

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего
образования

«Национальный исследовательский технологический университет «МИСИС»

(НИТУ «МИСИС»)

ИНСТИТУТ

Институт компьютерных наук

КАФЕДРА

инфокоммуникационных технологий

НАПРАВЛЕНИЕ

09.03.02 — Информационные системы и технологии

ДОМАШНЕЕ ЗАДАНИЕ № 1

по дисциплине «Оптимизация клиент-серверных приложений»

на тему: «Разработка сервиса по индивидуальному варианту и измерение исходных
показателей производительности»

Вариант 3

Студент группы БИСТ-24-ПО-3 _____ Скориков К. А.

Преподаватель _____ Жердев А. А., к.т.н., доцент

Москва, 2026

СОДЕРЖАНИЕ

ЗАДАНИЕ И ВАРИАНТ	3
ВВЕДЕНИЕ	3
1 Сервис, стенд и данные	4
1.1 Сервис и контракт	4
1.2 Модульные тесты	6
1.3 Условия испытания	6
1.4 Наполнение базы данных	7
2 Измерения	7
2.1 Время ответа операций	7
2.2 Разложение времени между приложением и базой данных	9
3 Гипотезы об узких местах	10
3.1 Гипотеза 1	10
3.2 Гипотеза 2	11
3.3 Гипотеза 3	11
ЗАКЛЮЧЕНИЕ	12
ПРИЛОЖЕНИЕ А Воспроизведение результатов	12

ЗАДАНИЕ И ВАРИАНТ

Таблица 1 – Вариант и стенд

Параметр	Значение
Вариант и предметная область	3 — запись на приём к специалисту
Префикс	kirill_skorikov
Язык и каркас серверной части	Python 3.12.14, Flask 3.1.2
Доступ к данным	psycopg 3.2.12; пул psycopg_pool 3.2.8, 1–4 соединения
СУБД	PostgreSQL 16.15, Windows x64
Репозиторий	https://github.com/ssskorrr/dz_1_oksp

Сервис хранит расписание специалистов в виде слотов и позволяет выбрать услугу, найти свободное время и забронировать его. Бронирование отклоняется при недостаточной длительности, несовпадении специализации или пересечении с активной записью. Отмена освобождает время для новой записи. Сводка рассчитывает загрузку специалистов и долю отменённых записей за заданный период.

ВВЕДЕНИЕ

Оптимизация веб-сервиса требует исходной точки сравнения: без измерения неизвестно, какие операции замедляются при росте данных и на что расходуется время. Медиана описывает типичную задержку, а 95-й процентиль и максимум позволяют увидеть длинные ответы. Разложение серверного времени на обращения к базе и оставшуюся обработку помогает сформулировать проверяемые причины замедления.

Цель работы — разработать сервис записи к специалисту и измерить исходные показатели его производительности на двух объёмах данных.

Для достижения цели необходимо решить следующие задачи:

1. реализовать сервер, клиент и авторизацию для варианта 3.
2. описать контракт восьми операций API.
3. покрыть бизнес-правила и операции тестами.
4. воспроизводимо наполнить PostgreSQL малым и рабочим объёмами.
5. измерить время ответа после прогрева.
6. разложить серверное время на обращения к БД и прочую обработку.

7. сформулировать не менее трёх гипотез об узких местах.
8. исправить выявленные ошибки и повторить измерения.

1 Сервис, стенд и данные

1.1 Сервис и контракт

Сервер реализован на Flask, клиент — HTML и JavaScript без сборки. Авторизация HTTP Basic выполняется на каждой операции API под учебной учётной записью demo. В интерфейсе представлены список записей с фильтром и постраничным выводом, карточка со связанными сущностями, подбор и бронирование слотов, сводка. В PostgreSQL созданы четыре предметные таблицы и отдельная таблица авторизации.

Таблица 2 – Операции программного интерфейса

Метод и путь	Параметры	Ответ	Ошибки
GET /api/services	нет	items: услуги и длительность	401
GET /api/specialists	нет	items: специалисты и специализация	401
GET /api/bookings	page, size, status	items, total, page, size	401, 422
GET /api/bookings/{id}	id	запись и связанные сущности	401, 404
GET /api/slots	service_id, specialist_id, from, to, page, size	свободные items, total, page, size	401, 404, 422
GET /api/summary	from, to	items, total_bookings, cancelled	401, 422
POST /api/bookings	JSON: slot_id, service_id	201: созданная запись	401, 404, 409, 422
POST /api/bookings/{id}/cancel	id	200: отменённая запись	401, 404

Полный контракт, поля и граничные условия приведены в README.md. Период полуоткрытый [from,to); по умолчанию 01.11.2026–01.02.2027. Запись занимает выделенный слот, услуга должна укладываться в его длительность. Повторная отмена идиempотентна. Защита от конкурентных бронирований обеспечивается транзакционной блокировкой по специалисту и повторной проверкой пересечений после получения блокировки. Ответы не кешируются.

Запись к специалисту

kinill_skorikov · курсовая работа - вариант 3 · PostgreSQL

Логин

demo

Пароль

....

Войти

Вход выполнен

Операция выполнена · 317.150 мс · запросов к БД: 2

Свободные слоты и бронирование

Услуга

Услуга 1 · 30 мин.

Специалист

Специалист 1

С

01.11.2026

До

01.02.2027

Найти слоты

Мои записи

Статус

Активные

Показать

№	Специалист	Услуга	Начало	Статус	Действия
1	Специалист 1	Услуга 1	Sun, 01 Nov 2026 09:00:00 GMT	active	Карточка Отменить
2	Специалист 2	Услуга 2	Sun, 01 Nov 2026 09:00:00 GMT	active	Карточка Отменить

Рисунок 1 – Авторизация и список записей на рабочем объёме

Выберите запись в списке.

Сводка загрузки за выбранный период

Обновить сводку

Записей за период: 50070; отменено: 10070

Специалист	Записей	Отменено	Загрузка	Доля отмен
Специалист 1	320	70	12.5%	21.9%
Специалист 2	250	0	25.0%	0.0%
Специалист 3	250	0	12.5%	0.0%
Специалист 4	250	0	25.0%	0.0%
Специалист 5	250	250	0.0%	100.0%
Специалист 6	250	0	12.5%	0.0%
Специалист 7	250	0	25.0%	0.0%
Специалист 8	250	0	12.5%	0.0%
Специалист 9	250	0	25.0%	0.0%
Специалист 10	250	250	0.0%	100.0%
Специалист 11	250	0	12.5%	0.0%

Рисунок 2 – Сводка загрузки после серии измерений

5

1.2 Модульные тесты

Команда: `python -m pytest -q`. Итог: 22 passed in 0.95s. Шесть тестов проверяют бизнес-правила: пересечение и смежность интервалов, длительность услуги, долю отмен и пустой знаменатель. Шестнадцать тестов проверяют API: авторизацию, формат карточки и списка, пагинацию, ошибки параметров, бронирование, отмену, свободные слоты, сводку и конкуренцию.

Листинг 1 – Примеры тестов

```
def test_adjacent_slots_do_not_overlap():
    assert not overlaps(T, T+timedelta(hours=1),
                        T+timedelta(hours=1), T+timedelta(hours=2))

def test_pagination_contract(client):
    r = client.get("/api/bookings?status=all&page=1&size=7",
                  headers=AUTH)
    assert r.status_code == 200
    assert len(r.json["items"]) == 7
    assert r.json["total"] == 300
```

1.3 Условия испытания

Таблица 3 – Условия испытания

Что	Значение
Машина	AMD Ryzen 5 2600, 6 ядер / 12 потоков, 16 ГБ ОЗУ, Windows x64
Среда выполнения	Python 3.12.14; Flask threaded development server; приложение 127.0.0.1:8089; PostgreSQL 127.0.0.1:55439
Учётная запись	demo; HTTP Basic; одинаковая во всех сериях
Число повторов	30 последовательных запросов на операцию и объём; конкурентность 1
Прогрев	5 запросов на операцию, не входят в статистику
Измеритель	benchmark.py; perf_counter; прямое HTTP-подключение без системного прокси
Настройки PostgreSQL	jit=off; max_parallel_workers_per_gather=0; shared_buffers=128MB; work_mem=4MB
Пул и индексы	Пул 1–4 соединения; только РК; дополнительных индексов нет

Дата испытания — 8 октября 2026 года. Оба объёма измерены на одном компьютере с одинаковой конфигурацией и исходниками. Перед каждой серией выполнены два одинаковых наполнения с совпадением числа строк и отпечатка bookings. Полные версии зависимостей сохранены в evidence/versions.txt. Параллельные исполнители PostgreSQL отключены после аварии процесса Windows 0xC0000005; аварийные серии исключены, оба объёма измерены заново. Выводы относятся к этому локальному стенду, а не к промышленному развёртыванию.

1.4 Наполнение базы данных

Команды: `python seed.py small`; `python seed.py work`. Программа применяет `schema.sql` и пересоздаёт данные только в выделенной схеме `kirill_skorikov`. Даты, распределение сущностей и статусы детерминированы. Отпечатки `bookings`: малый `28740c170998e764be6cf374f1022f0f`; рабочий `8f762755f2bb422c9cb106774eedffc1`; при повторных запусках совпали.

Таблица 4 – Объём наполнения до измерений

Таблица	Малое	Рабочее	Рост
app_users	1	1	1.00
services	10	30	3.00
specialists	10	200	20.00
slots	500	200000	400.00
bookings	300	50000	166.67

Основная операционная таблица `bookings` содержит 300 и 50 000 записей. Из них 20% отменены. Во время измерений POST дополнительно создаёт 35 записей, а подготовка серии отмены — ещё 35: всего 70, включая прогрев. Все они отменены после измеряемого действия; после полного набора остаются 370 и 50 070 строк. Изменения не входят во времена других GET-операций, поскольку GET измеряются первыми. На рисунке 2 поэтому показаны 50 070 записей и 10 070 отменённых.

2 Измерения

2.1 Время ответа операций

Длительность HTTP измеряется от начала запроса до полного чтения JSON. Для p95 используется ближайший ранг $\text{ceil}(0,95 \times 30) = 29$. В серии бронирования последующая отмена не входит во время; в серии отмены предварительное создание записи не входит во время.

Таблица 5 – Медиана и 95-й процентиль времени ответа в миллисекундах

Операция	p50 малое	p95 малое	p50 рабочее	p95 рабочее	Рост p50
GET /services	5.30	26.28	5.72	22.41	1.08
GET /specialists	4.58	25.42	5.90	25.59	1.29
GET /bookings	7.23	23.32	17.42	35.48	2.41
GET /bookings/{id}	6.15	25.81	5.89	25.48	0.96
GET /slots	7.65	29.29	176.37	243.89	23.06
GET /summary	8.19	29.62	265.25	292.14	32.39
POST /bookings	9.08	22.52	98.77	119.17	10.88
POST /bookings/{id}/cancel	5.80	26.59	4.88	27.08	0.84

Таблица 6 – Максимальное время ответа в миллисекундах

Операция	Максимум малое / рабочее
GET /api/services	29.26 / 25.45
GET /api/specialists	26.97 / 27.00
GET /api/bookings	30.28 / 35.88
GET /api/bookings/{id}	26.50 / 27.97
GET /api/slots	32.04 / 299.16
GET /api/summary	31.26 / 311.94
POST /api/bookings	28.38 / 125.59
POST /api/bookings/{id}/cancel	27.63 / 27.23

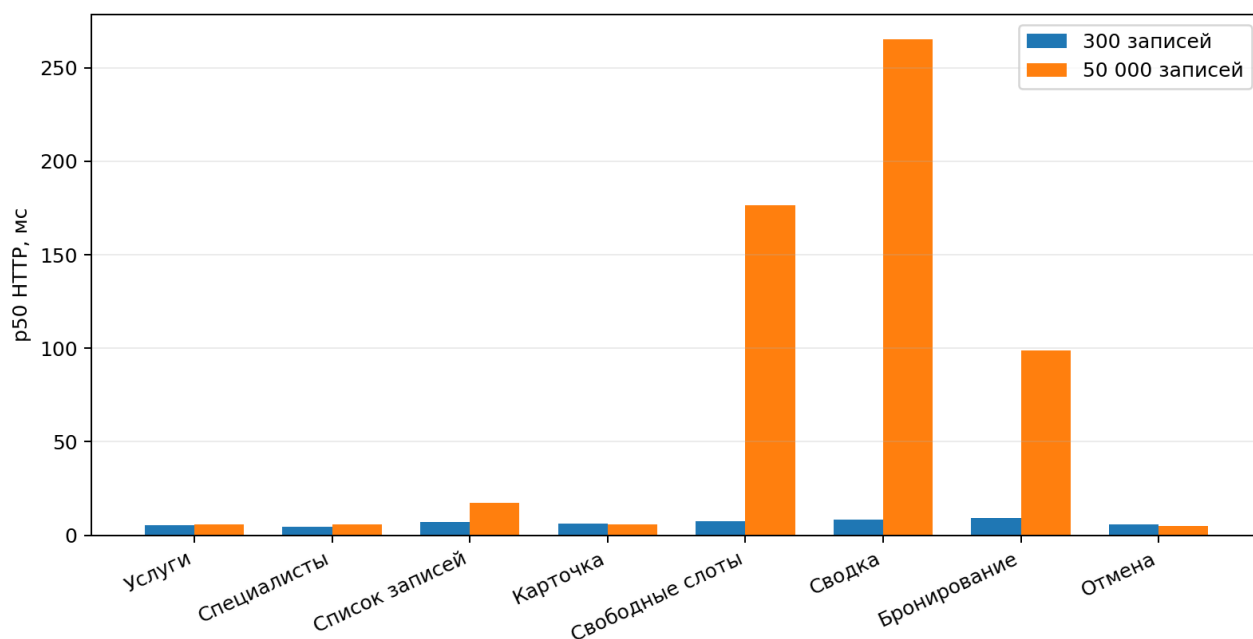


Рисунок 3 – Сравнение медианы времени HTTP на двух объёмах

Наиболее заметный рост показали свободные слоты, сводка и бронирование. Рост основной таблицы составил 166,67 раза; рост времени перечисленных операций меньше этого коэффициента, то есть по двум точкам нельзя утверждать сверхлинейную сложность. Тем не менее именно они формируют самые длительные ответы рабочего объёма. Список записей также замедлился из-за подсчёта общего количества и выборки с фильтром.

Услуги, специалисты, карточка записи и отмена остаются в диапазоне единиц миллисекунд по медиане, с ростом менее 1,5 раза. Они исключены из дальнейшего разбора узких мест. Двух точек недостаточно для строгого определения асимптотики, поэтому выводы сформулированы как гипотезы для следующего этапа.

2.2 Разложение времени между приложением и базой данных

В app.py общий таймер охватывает серверную обработку и сериализацию JSON. Database.query измеряет cursor.execute и чтение результата; его время суммируется по запросам, число запросов считается отдельно. Для разложения используются средние тридцати измерений, чтобы компоненты складывались точно. Серверное время отличается от внешнего HTTP-времени в таблице 5.

Таблица 7 – Куда уходит серверное время на рабочем объёме

Операция	Всего мс	БД мс	Остальное мс	Запросов	Вывод
GET /bookings	13.73	12.16	1.57	3	Преобладает БД
GET /slots	178.80	177.35	1.44	5	Преобладает БД
GET /summary	259.27	257.74	1.52	2	Преобладает БД
POST /bookings	86.32	84.34	1.98	7	Преобладает БД

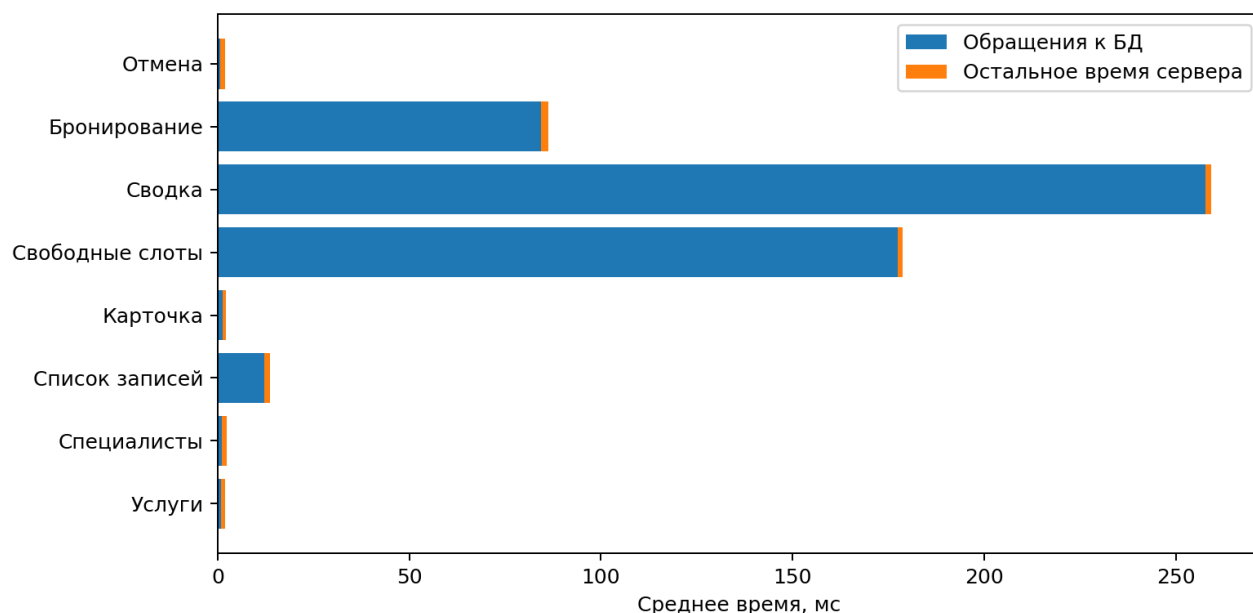


Рисунок 4 – Среднее время обращений к БД и остальной обработки

Время БД включает транспорт и декодирование строк, поэтому это длительность обращения к PostgreSQL, а не чистое время его CPU. Остаток включает пул, SET search_path, commit, Python и JSON. Число SQL-запросов фиксировано на операцию и не растёт с числом возвращаемых записей: признаков N+1 в реализованных операциях нет. Во всех четырёх отобранных операциях преобладают обращения к БД.

3 Гипотезы об узких местах

3.1 Гипотеза 1

Операция: GET /api/slots.

Основание: p50 7.65 → 176.37 мс, рост 23.06 раза; среднее серверное время 178.80 мс, обращения к БД 177.35 мс, 5 SQL-запросов.

Предполагаемая причина: Подсчёт и выборка свободных слотов повторяют проверку NOT EXISTS по активным бронированиям и пересечениям времени. Нет индекса по специалисту

и времени; PostgreSQL приходится читать существенную часть slots и bookings.

Проверка гипотезы: На этапе 2 изучить EXPLAIN ANALYZE и профилирование; на этапе 3 проверить индексы и кеш свободных слотов с корректным сбросом после записи и отмены.

3.2 Гипотеза 2

Операция: GET /api/summary.

Основание: p50 8.19 → 265.25 мс, рост 32.39 раза; среднее серверное время 259.27 мс, обращения к БД 257.74 мс, 2 SQL-запросов.

Предполагаемая причина: Сводка агрегирует все слоты и бронирования периода, объединяет услуги и специалистов. При рабочем объёме обрабатываются 200 000 слотов и 50 000 записей; стоимость растёт из-за чтения, соединений и агрегирования.

Проверка гипотезы: Сопоставить планы выполнения и объём прочитанных страниц; затем проверить оптимизацию агрегирующих запросов или кеширование с доказательством корректности.

3.3 Гипотеза 3

Операция: POST /api/bookings.

Основание: p50 9.08 → 98.77 мс, рост 10.88 раза; среднее серверное время 86.32 мс, обращения к БД 84.34 мс, 7 SQL-запросов.

Предполагаемая причина: Перед вставкой проверяются пересечения по специалисту среди активных бронирований; специализированных индексов нет. Время включает ожидание advisory-блокировки, но при конкурентности 1 конкуренции нет: основная гипотеза — чтение для проверки конфликтов.

Проверка гипотезы: Измерить отдельную проверку конфликтов, план соединения bookings со slots, затем проверить индексирование; блокировку сохранять для корректности.

Исправленные ошибки: возврат соединения при повторном teardown тестового клиента выполнялся дважды; теперь соединение извлекается из g через ror, возврат выполняется один раз. При потере соединения rollback больше не скрывает первоначальную ошибку, соединение возвращается в пул через finally. Прямое HTTP-подключение измерителя настроено без системного прокси. Эти исправления касаются корректности стенда и обработки соединений. Итоговые тесты и оба набора замеров получены после исправлений; прежние серии не использованы.

ЗАКЛЮЧЕНИЕ

Цель достигнута: разработан сервис записи к специалисту и измерены исходные показатели на двух объёмах PostgreSQL.

Для этого решены следующие задачи:

- реализованы сервер, клиент, авторизация и три обязательных экрана.
- описаны восемь операций API.
- выполнены 22 теста, включая конкурентное бронирование и освобождение слота.
- подготовлены воспроизводимые наполнения на 300 и 50 000 записей.
- сняты по 30 повторов восьми операций на каждом объёме после пяти прогревочных запросов.
- разложены времена четырёх отобранных операций.
- сформулированы три гипотезы с численными основаниями.
- исправлены ошибки обработки соединений и стенда, измерения выполнены повторно.

Самая длительная операция рабочего набора — сводка: p50 265.25 мс; свободные слоты — 176.37 мс. Разложение показало преобладание времени обращений к PostgreSQL. Следующий этап должен подтвердить причины планами выполнения и нагрузочным испытанием; преждевременные индексы и кеша в первый этап не добавлялись.

ПРИЛОЖЕНИЕ А Воспроизведение результатов

Из корня проекта после настройки DATABASE_URL: `python -m pytest -q`; `python app.py` (отдельный терминал); `python benchmark.py small`; `python benchmark.py work`. Полный порядок установки, контракт и настройки — README.md. Отдельные наблюдения — `evidence/small.json` и `evidence/work.json`; версии — `evidence/versions.txt`; результат тестов — `evidence/tests.txt`. Архив включает исходный код, данные эксперимента и инструкции.